

FIFO Page Replacement

```
#include <stdio.h>
#include <stdlib.h>
#define FRAME_SIZE 3

int findPage(int page, int *memory, int frameSize) {
    for (int i = 0; i < frameSize; i++) {
        if (memory[i] == page)
            return 1;
    }
    return 0;
}

void replacePage(int page, int *memory, int frameSize, int *fifoQueue,
int *queuePointer) {
    int oldestPage = fifoQueue[*queuePointer];
    fifoQueue[*queuePointer] = page;
    *queuePointer = (*queuePointer + 1) % frameSize;
    for (int i = 0; i < frameSize; i++) {
        if (memory[i] == oldestPage) {
            memory[i] = page;
            break;
        }
    }
}

int main() {
    int pages[] = {1, 2, 3, 4, 5, 2, 1, 6, 7, 8, 7, 8, 9, 7, 8, 9, 5, 4,
5, 4};
    int numPages = sizeof(pages) / sizeof(pages[0]);
    int memory[FRAME_SIZE] = {0};
    int fifoQueue[FRAME_SIZE] = {0};
    int queuePointer = 0;
```

```
int pageFaults = 0;

for (int i = 0; i < numPages; i++) {
    int page = pages[i];
    if (!findPage(page, memory, FRAME_SIZE)) {
        pageFaults++;
        if (i < FRAME_SIZE) {
            memory[i] = page;
            fifoQueue[i] = page;
        } else {
            replacePage(page, memory, FRAME_SIZE, fifoQueue,
&queuePointer);
        }
    }
}

printf("Total page faults: %d\n", pageFaults);
return 0;
}
```

Output:

```
[Running] cd "g:\OS\lab8\" && gcc tempCodeRunnerFile.c -o tempCodeRunnerFile && "g:\OS\lab8\"tempCodeRunnerFile
Total page faults: 13
```

OPTIMAL PAGE REPLACEMENT – OPR

```
#include <stdio.h>
#include <stdlib.h>
#define FRAME_SIZE 3

int findPage(int page, int *memory, int frameSize) {
    for (int i = 0; i < frameSize; i++) {
        if (memory[i] == page)
            return 1;
    }
    return 0;
}

int findFarthestPage(int *pages, int numPages, int *memory, int
frameSize, int currentIndex) {
    int farthestIndex = -1;
    int farthestDistance = -1;

    for (int i = 0; i < frameSize; i++) {
        int nextPageIndex = currentIndex;
        int page = memory[i];
        while (++nextPageIndex < numPages) {
            if (pages[nextPageIndex] == page) {
                if (nextPageIndex > farthestDistance) {
                    farthestDistance = nextPageIndex;
                    farthestIndex = i;
                }
                break;
            }
        }
        if (nextPageIndex == numPages)
            return i;
    }
}
```

```
        return farthestIndex;
    }

    int main() {
        int pages[] = {1, 2, 3, 4, 5, 2, 1, 6, 7, 8, 7, 8, 9, 7, 8, 9, 5, 4,
5, 4}; // Sequence of page accesses
        int numPages = sizeof(pages) / sizeof(pages[0]);

        int memory[FRAME_SIZE] = {0};
        int pageFaults = 0;

        for (int i = 0; i < numPages; i++) {
            int page = pages[i];
            if (!findPage(page, memory, FRAME_SIZE)) {
                pageFaults++;
                if (i < FRAME_SIZE) {
                    memory[i] = page;
                } else {
                    int replaceIndex = findFarthestPage(pages, numPages,
memory, FRAME_SIZE, i);
                    memory[replaceIndex] = page;
                }
            }
        }

        printf("Total page faults: %d\n", pageFaults);

        return 0;
    }
```

Output:

```
[Running] cd "g:\OS\lab8\" && gcc tempCodeRunnerFile.c -o tempCodeRunnerFile &&  
"g:\OS\lab8\"tempCodeRunnerFile
```

```
Total page faults: 11
```

```
[Done] exited with code=0 in 2.732 seconds
```

Least Recently Used (LRU)

```
#include <stdio.h>

#include <stdlib.h>

#define FRAME_SIZE 3

int findPage(int page, int *memory, int frameSize) {
    for (int i = 0; i < frameSize; i++) {
        if (memory[i] == page)
            return i;
    }
    return -1;
}

void shiftRight(int *memory, int index, int frameSize) {
    for (int i = index; i > 0; i--) {
        memory[i] = memory[i - 1];
    }
}

void replaceLRUPage(int page, int *memory, int frameSize) {
    shiftRight(memory, frameSize - 1, frameSize);
    memory[0] = page;
}

int main() {
    int pages[] = {1, 2, 3, 4, 5, 2, 1, 6, 7, 8, 7, 8, 9, 7, 8, 9, 5, 4, 5, 4};
    int numPages = sizeof(pages) / sizeof(pages[0]);
```

```
int memory[FRAME_SIZE] = {0};

int pageFaults = 0;

for (int i = 0; i < numPages; i++) {
    int page = pages[i];
    int index = findPage(page, memory, FRAME_SIZE);
    if (index == -1) {
        pageFaults++;
        if (i < FRAME_SIZE) {
            shiftRight(memory, i, FRAME_SIZE);
            memory[0] = page;
        } else {
            replaceLRUPage(page, memory, FRAME_SIZE);
        }
    } else {
        shiftRight(memory, index, FRAME_SIZE);
    }
}

printf("Total page faults: %d\n", pageFaults);

return 0;
}
```

Output:

```
[Running] cd "g:\OS\lab8\" && gcc tempCodeRunnerFile.c -o  
tempCodeRunnerFile && "g:\OS\lab8\"tempCodeRunnerFile
```

```
Total page faults: 14
```

```
[Done] exited with code=0 in 1.3 seconds
```